به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴)

دانشگاه صنعتی شریف — تمرین سوم

مدرس: مهدی صفرنژاد — انتشار: ۲۹ اسفند ۹۸

تفکر الگوریتمی

زمان تحویل این تمرین (شامل بخش نظری و بخش عملی) شنبه ۱۶ اسفند ۹۹ ساعت ۱:۰۰ بامداد (یک ساعت پس از نیمه‌شب جمعه) می‌باشد. به توضیحات سایت درس دربارهٔ تمرین‌ها، تحویل آن و تأخیر توجه کنید. تمرین‌ها از سایت کوئرا تحویل گرفته می‌شوند. قسمت‌های نظری به صورت فایل PDF تحویل گرفته می‌شوند که این فایل یا باید با حروف‌چینی رایانه‌ای تولید شده باشد و یا اگر دست‌نوشته است، توسط نرم‌افزارهایی مثل CamScanner یا Office Lens و یا با دستگاه اسکنر تصویربرداری شده باشد.

---

۱ تمرینات تئوری

سؤال ۱. [۱۰ نمره] مقادیر ۱۰، ۱۲، ۱۴، ۴، ۲، ۱۶، ۱۸ و ۱۱ را به ترتیب از راست به چپ در درخت های خالی Red Black و AVL اضافه کنید. سپس مقادیر ۲، ۱۴ و ۱۰ را به ترتیب از راست به چپ، از دو درخت حذف کنید. درخت حاصل پس از هر عمل حذف و اضافه را رسم کنید.

پاسخ: پاسخ. برای مشاهده‌ی این نتایج می‌توانید از لینک‌های زیر استفاده کنید.

https://www.cs.usfca.edu/~galles/visualization/RedBlack.html

https://www.cs.usfca.edu/~galles/visualization/AVLtree.html

سؤال ۲. [۲۰ نمره] درختی n راسی و ریشه دار داریم. رئوس با اعداد ۱ تا n شماره گذاری شده‌اند و راس شماره ۱ ریشه‌ی درخت است. علاوه بر عدد، هر راس یک مقدار ویژه هم دارد. به ازای یک راس تمام مقدار ویژه های رئوس زیر درختش را در نظر بگیرید. مینیمم تفاوت بین مقدار ویژه‌ها، امتیاز ویژه‌ی آن راس است. حال روشی ارائه دهید که بتوانیم امتیاز ویژه‌ی همه‌ی راس‌ها را در $O(n\log(n)^۲)$ بدست آوریم.

پاسخ: پاسخ. برای این کار، تابعی به شکل $f(v)$ تعریف می‌کنیم که این تابع، یک درخت AVL ارائه می‌دهد که در آن تمام اعداد رئوس زیردرخت راس v در آن آمده است. هم چنین امتیاز ویژه‌ی راس v را نیز می‌دهد. حال می‌خواهیم روشی برای این تابع ارائه دهیم. برای ساخت درخت AVL ، کافی است یک درخت AVL بسازیم که شامل عدد راس v است. سپس درخت‌های AVL بچه‌های راس v را بدست آوریم و به ترتیب آن‌ها را با درخت AVL راس v ادغام کنیم.

برای پیدا کردن امتیاز ویژه هم می‌دانیم دو راس با اختلاف مینیمم، یا هر دو در زیردرخت یکی از بچه‌های راس v هستند که کافی است امتیاز ویژه‌ی آن بچه را داشته باشیم. در غیر اینصورت می‌دانیم هنگام ادغام کردن، باید مینیمم اختلاف بین اعداد دو درخت AVL را نیز بدست آوریم. پس برای محاسبه‌ی $f(v)$ ابتدا تابع f را برای همه‌ی بچه‌های راس v صدا می‌زنیم تا مواردی که در بالا نیاز است، بدست بیاید. حال برای محاسبه‌ی امتیاز ویژه و درخت AVL به صورت زیر عمل می‌کنیم.

فرض کنیم درخت AVL راس i را با AVL_i نشان دهیم. u را بچه‌ی v در نظر می‌گیریم و می‌خواهیم دو درخت AVL_v, AVL_u را ادغام کنیم. برای این کار، اگر AVL_v راس های کمتری نسبت به AVL_u داشت، تک تک اعضای AVL_v را به AVL_u اضافه می‌کنیم. در غیر اینصورت برعکس عمل می‌کنیم. در واقع درخت با راس‌های کمتر را به درخت با راس‌های بیشتر اضافه می‌کنیم. درخت حاصل را به راس v نسبت می‌دهیم.

هر بار که یک عضو را اضافه می‌کنیم، برای این که ببینیم با اعداد درخت دیگر مینیمم اختلافی که می‌سازد چقدر است، بزرگترین عدد کوچکتر یا مساوی و کوچکترین عدد بزرگتش در AVL ای که به آن اضافه می‌شود هم بدست می‌آوریم. مینیمم اختلاف با این دو مقدار، مینیمم مقدار اختلافی است که می‌تواند با اعداد این درخت بسازد(چرا؟). در نتیجه وقتی زیردرخت v را با کل بچه‌هایش ادغام کردیم، امتیاز ویژه‌اش مینیمم امتیاز ویژه‌های بچه‌هایش و اعداد بدست آمده در ادغام هاست.

حال برای حل مسئله کافی است تنها یک بار $f(۱)$ را صدا کنیم. در نتیجه‌ی این کار f تمام رئوس صدا زده می‌شود و امتیاز ویژه‌ی هر راسی نیز محاسبه می‌شود. حال این راه را تحلیل اردر می‌کنیم. یک راس دلخواه را در نظر بگیرید. می‌خواهیم بررسی کنیم این عدد چند بار به یک درخت AVL اضافه می‌شود. وقتی دو AVL را ادغام می کنیم، اعداد AVL کوچکتر به AVL بزرگتر اضافه می‌شوند. درنتیجه AVL نهایی از ادغام این دو AVL ، حداقل دو برابر AVL کوچکتر راس دارد. پس هر بار که یک عدد به یک درخت AVL اضافه شود، تعداد راس های AVL جدیدی که به آن اضافه می‌شود نسبت به AVL قبلی‌اش حداقل دو برابر شده است. چون هیچ AVL ای بیشتر از n راس ندارد و هر بار که یک درخت AVL به درخت AVL دیگری اضافه شود دیگر با آن کاری نداریم، می‌فهمیم عدد هر راسی حداکثر $\log(n)$ بار به یک درخت AVL اضافه می‌شود. هم چنین یافتن بزرگترین عدد کوچکتر یا مساوی و کوچکترین عدد بزرگتر نیز از $O(\log(n))$ است. پس کل عملیات ها از اردر زیر است:

$$Time = O(n * \log(n) * \log(n)) = O(n\log(n)^{۲})$$

سؤال ۳. [۱۷ نمره] یک آرایه‌ی n تایی داریم که همه‌ی اعداد آن در ابتدا یک هستند. q درخواست به شکل زیر داده می شود:

اعداد بازه‌ی $[l_i, r_i]$ را در عدد p که یک عدد اول است، ضرب کنید. هم چنین می‌دانیم که $۱ \le l_i \le r_i \le n$ و $۲ \le p \le n$ برقرار است.

حال پس از اعمال q درخواست، عدد i ـ ام آرایه به a_i تبدیل شده است. در آخر امتیاز آرایه برابر مجموع تعداد مقسوم علیه‌های اعداد آرایه است. حال شما با فرض داشتن n و q درخواست، باید در $O(n+q)$ امتیاز آرایه را بدست آورید. هم چنین می‌توانید فرض کنید که هیچ عددی پس از اعمال درخواست ها، بزرگتر از عدد ثابت C نخواهد شد.

پاسخ: پاسخ. اگر بدانیم هر خانه از آرایه، قرار است در $p_i{}^{x_i}$ ضرب شود که p_i برابر با i امین عدد اول است و توان این عامل نیز برابر x_i باشد، تعداد مقسوم علیه‌های عددی که در این خانه نوشته می‌شود برابر است با: $(x_۱ + ۱)(x_۲ + ۱)(x_۳ + ۱)...$. درنتیجه برای هر خانه می‌خواهیم بدست بیاوریم چه اعداد اولی توانی بیشتر از ۰ دارند.

حال اگر یک درخواست را در نظر بگیرید، عوامل اول یک توان را در یک بازه به علاوه‌ی یک می‌کند. پس باید روشی بیابیم که بتوانیم به وسیله‌ی آن به گونه‌ای این کار را انجام دهیم. برای همین برای هر خانه از آرایه، دو لیست در نظر میگیریم. این دو لیست را برای خانه‌ی i ام، $start_i$ و end_i می‌نامیم. حال به ازای هر i ، برای درخواست i ام، p_i را در $start_{l_i}$ و end_{r_i} اضافه می‌کنیم. این کار نشان‌دهنده‌ی این است که در خانه‌ی l_i یک ضرب در p_i آغاز می‌شود و در خانه‌ی r_i نیز تمام می‌شود.

حال یک آرایه به نام power و به طول C در نظر می‌گیریم که در ابتدا همه یک است(خانه‌ی i ام آن نشان‌دهنده‌ي توان عدد i به

علاوه‌ی یک، در تجزیه یک عدد است. اگر i اول نباشد این مقدار همیشه یک می‌ماند). می‌خواهیم این آرایه را به گونه‌ای آپدیت کنیم که برای هر خانه از جدول، بفهمیم توان های عوامل اول چقدر است. برای این کار به شکل زیر عمل می‌کنیم:

ابتدا یک متغیر به نام cnt را که حاصل ضرب اعداد آرایه است برابر یک قرار می‌دهیم. به ازای $i = ۱, ۲, ..., n$ کار های زیر را به ترتیب انجام می‌دهیم.

ـ ابتدا به ازای هر عضو لیست $start_i$ مثل p باید توان این عامل را زیاد کنیم. برای این کار cnt را تقسیم بر توان کنونی ($power_p$) می‌کنیم، $power_p$ را یکی زیاد می‌کنیم. cnt را در $power_p$ ضرب می‌کنیم.

ـ عددی که cnt دارد، برابر تعداد مقسوم علیه های عدد i ام آرایه است.

ـ به ازای هر عضو لیست end_i مثل p باید توان این عامل را کم کنیم. برای این کار cnt را تقسیم بر توان کنونی می‌کنیم، توان کنونی را کم می‌کنیم، cnt را در توان جدید ضرب می‌کنیم.(همانند گام یک)

تمام عملیات های ریاضی ما از $O(۱)$ خواهد بود زیرا اعداد از عددی ثابت که در صورت سوال ذکر شده بیشتر نمی‌شوند. هم چنین به ازای هر درخواست داریم از $O(۱)$ کار انجام می‌دهیم و کلا هم n خانه را بررسی می‌کنیم. پس اردر این الگوریتم از $O(n+q)$ است.

سؤال ۴. [۱۸ نمره] فرض کنید که n ماهی داریم. ماهی i–ام وزنی برابر S_i و قدرت حمله‌ای برابر E_i دارد به صورتی که می‌دانیم: $E_i \leq S_i$. ماهی i–ام به ماهی j–ام زور می‌گوید اگر که داشته باشیم: $E_i > S_j$. حال می‌خواهیم بیشترین تعداد ماهی را انتخاب کنیم به صورتی که بین ماهی های انتخاب شده، هیچ یک از ماهی ها به ماهی دیگری زور نگوید. تعداد این ماهی ها را در $O(n\log(n))$ بیابید.

پاسخ: پاسخ. برای حل این سوال، فرض می‌کنیم ماکسیمم قدرت حمله برابر E باشد. بنابراین تمام ماهی‌هایی که در مجموعه هستند وزنی بیشتر یا مساوی E دارند. زیرا در غیر این‌صورت، ماهی با قدرت حمله ماکسیمم به آنها زور می‌گوید. هم چنین اگر چنین شرطی برقرار باشد، هیچ دو ماهی‌ای به هم زور نمی‌گویند. پس کافی است ماکسیمم قدرت حمله را مشخص کنیم، سپس از بین ماهی هایی که قدرت حمله‌ای کمتر یا مساوی قدرت حمله‌ی ماکسیمم دارند، تمام ماهی هایی که وزنی بیشتر یا مساوی قدرت حمله‌ی ماکسیمم را دارند انتخاب کنیم. برای این کار ماهی ها را بر حسب قدرت حمله‌شان مرتب می‌کنیم و الگوریتم زیر را اجرا می‌کنیم:

فرض کنید ماهی‌هایی که بر حسب قدرت حمله‌شان مرتب شده‌اند، شماره های ۱ تا n را به ترتیب دارند(۱ کمترین زور، n بیشترین زور). به ازای $i = ۱, ۲, ..., n$ مراحل زیر را اجرا می‌کنیم.

ـ وزن ماهی ام–i را به یک درخت AVL اضافه می‌کنیم.

ـ تعداد اعداد درخت AVL را که از قدرت حمله‌ی ماهی ام–i بیشتر است را بدست می‌آوریم.

از بین n عددی که بدست آورده ایم، بیشترین عدد، بیشترین تعداد ماهی‌ای است که می‌توانیم دور هم جمع کنیم به صورتی که هیچ دوتایی به هم زور نگویند.

برای شمردن تعداد اعداد بزرگتر یا مساوی در درخت AVL، می‌دانیم می‌توانیم برای هر راس، تعداد اعضای زیر درخت هایش را نگه داریم. سپس اگر بخواهیم اعداد بزرگتر یا مساوی x را حساب کنیم، x را در درخت جستجو می‌کنیم. هر بار که به بچه‌ی چپی برویم(کوچکتر یا مساوی راسی که در آن هستیم)، میفهمیم آن راس و زیر درخت سمت راست همگی بزرگتر یا مساوی هستند و آنها را با مقداری که تاکنون بدست آورده‌ایم جمع می‌زنیم. اگر هم به بچه‌ی راستی(بزرگتر از راسی که در آن هستیم) برویم چیزی به

مقدارمان اضافه نمی‌کنیم. پس محاسبه‌ی این تعداد و هم چنین اضافه کردن اعداد به درخت AVL از $O(\log(n))$ است. پس کل راه از $O(n\log(n))$ است.

سؤال ۵. [۱۷ نمره] در یک کفاشی، n سایز مختلف کفش وجود دارد. مردی به داخل مغازه آمده و از مغازه‌دار می‌خواهد که یک کفش متناسب با اندازه‌ی پای او پیدا کند(می‌دانیم دقیقا یکی از کفش های مغازه برای مشتری مناسب است). مغازه‌دار می‌تواند در $O(۱)$ سایز دو کفش را مقایسه کند و بفهمد که کدام بزرگتر است. اما برای این که بفهمد پای مشتری بزرگ تر است یا یک کفش یا که این دو برابرند، نیاز دارد که کفش را پای مشتری بکند. مشتری از این کار خوشش نمی‌آید و هر بار که بخواهد کفشی را بپوشد، k ثانیه لفتش می‌دهد. مغازه‌دار هم که حوصله ندارد می‌خواهد سریع‌تر این کار را انجام دهد. راهی از اردر $O(n + k\log(n))$ به مشتری ارائه دهید. پاسخ: پاسخ. برای این کار از جستجوی دودویی استفاده می‌کنیم. برای این کار ابتدا تمام کفش ها را در نظر میگیریم. در $O(n)$ می‌توانیم میانه‌ی کفش ها از لحاظ اندازه را بدست آوریم. سپس کفش ها را به دو تکه‌ی بزرگتر از میانه و کوچکتر از میانه در $O(n)$ تقسیم می‌کنیم. حال امتحان می‌کنیم تا بفهمیم سایز کفش میانه و پای مشتری چه نسبتی به هم دارند. در صورت اندازه بودن کار تمام است. در غیر این صورت یا کفش مورد نظر در نیمه‌ی بزرگتر کفش هاست یا در نیمه‌ی کوچکتر کفش ها. سپس روی کفش های باقی مانده همین الگوریتم را تکرار می‌کنیم.

حال این راه را تحلیل می‌کنیم. حداکثر $\log(n)$ بار یک کفش را با پای مشتری مقایسه کرده ایم. زیرا هر بار که این کار را کنیم و کار تمام نشود، مجموعه کفش هایی که جواب در آنهاست نصف می‌شود و این کار حداکثر $\log(n)$ بار اتفاق می‌افتد. پس کل مقایسه‌های کفش‌ها و پای مشتری، از $O(k\log(n))$ است. حال پیدا کردن میانه‌ها را تحلیل می‌کنیم. فرض کنید پیدا کردن میانه‌ی n عدد حداکثر به $C * n$ عملیات نیاز دارد که C یک ثابت است. حال مجموع هزینه‌های پیدا کردن میانه‌ها را حساب می‌کنیم.

$$Time = C*n + C*\frac{n}{۲} + C*\frac{n}{۴} + ... + C*\frac{n}{۲^{\log(n)}} = C*n(۱ + \frac{۱}{۲} + ... + \frac{۱}{۲^{\log(n)}}) < C*n*۲ < C'*n$$

$$Time = C'*n$$

پس می‌فهمیم هزینه های یافتن کل میانه‌ها نیز از $O(n)$ است. پس اردر کل الگوریتم برابر است با: $O(n + k\log(n))$.

سؤال ۶. [۱۸ نمره] محمد که در حال مرور درس های گذشته‌اش بود، با یک سوال مواجه می‌شود. صورت سوال به شکل زیر است:

مسئله مرتب سازی را در نظر بگیرید. فرض کنید اعداد به صورت دقیق مشخص نیستند. در عوض برای هر عدد، یک بازه‌ی بسته روی اعداد حقیقی داریم که می‌دانیم آن عدد متعلق به آن بازه است. یعنی n بازه به صورت $[a_i, b_i]$ داریم، که در آن $a_i \leq b_i$. هدف این است که این بازه ها مرتب کنیم؛ یعنی یک جایگشت $\langle i_۱, i_۲, ..., i_n\rangle$ از بازه‌ها بسازیم به طوری که برای هر $j = ۱, ۲, ..., n$ مقادیر $c_i \in [a_{i_j}, b_{i_j}]$ موجود باشند که در نامساوی $c_۱ \leq c_۲ \leq ... \leq c_n$ صدق کنند. فرض کنید که هر بازه، حداقل با $d - ۱$ بازه‌ی دیگر هم‌پوشانی داشته باشد. الگوریتمی از $O(n\log(\frac{n}{d}))$ ارائه دهید که این n بازه با درجه‌ی d را مرتب کند. بدین ترتیب، اگر $d = \Theta(n)$ باشد، الگوریتم در زمان $O(n)$ اجرا خواهد شد. هم‌چنین اگر $d = \Theta(۱)$ باشد، الگوریتم در زمان $O(n\log(n))$ اجرا خواهد شد.

حال محمد که دیگر کلا این چیز ها یادش رفته است، از شما می‌خواهد که راه این سوال را به او بگویید. اگر هم محمد اشتباه می‌کند باید به او ثابت کنید که این کار شدنی نیست. پاسخ: پاسخ. این کار شدنی نیست. برای اثبات این کار، از برهان خلف استفاده می‌کنیم. فرض کنید که روشی با شرایط گفته شده وجود داشته باشد. می‌خواهیم یک الگوریتم مرتب‌سازی برای n عدد ارائه دهیم

که از $O(n)$ باشد. فرض کنید که n عدد به شکل $A_1, A_2, ..., A_n$ داریم. مینیمم این اعداد را A_x و ماکسیمم این اعداد را A_y می‌نامیم. حال n بازه متناظر این اعداد درست می‌کنیم که بازه‌ی i ام آنها به شکل $a_i = b_i = A_i$ تعریف می‌کنیم. به n بازه‌ای که ساختیم، n بازه‌ی دیگر اضافه می‌کنیم که بازه‌ی n+i ام بدین شکل است: $a_{n+i} = A_x - 1, b_{n+i} = A_y + 1$. حال این ۲n بازه را درنظر بگیرید. می‌دانیم d تعریف شده در الگوریتم حداقل برابر n است. طبق الگوریتم داده شده، می‌توانیم در اردر زیر این بازه‌ها را مرتب کنیم.

$$T = O(2n \log(\frac{2n}{d})) = O(n \log(2)) = O(n)$$

حال کافی است از جایگشت مرتب‌شده‌ی بازه‌ها، بازه‌های با شماره های $n+1$ تا $2n$ را حذف کنیم. بازه‌های باقی مانده، همان اعدادی هستند که می‌خواستیم آن‌ها را مرتب کنیم. پس توانستیم n عدد را در $O(n)$ مرتب کنیم. می‌دانیم این کار امکان‌پذیر نیست و مرتب‌سازی مقایسه‌ای باید از $\Omega(n \log(n))$ باشد.

سؤال ۷. [۳۳ نمره] یک روز طبق معمول، عیسی و موسی در حال گذران وقت بودند، که ناگهان موسی برای سرگرم کردن عیسی فکری به ذهنش رسید. او یک درخت دودویی تقریباً کامل کشید و از عیسی خواست که کمترین تعداد تعویض خانه‌ها برای اینکه درخت، یک درخت دودویی جستجو باشد را به او بگوید (عیسی می‌تواند مقدار هر دو رأس دلخواه از درخت را با یکدیگر عوض کند). عیسی هم که تا جواب سؤال موسی را نمی‌داد، ولکن نبود، تصمیم گرفت برنامه‌ای بنویسد تا دیگر موسی نتواند وقت او را با این کارهای بی‌ارزش بگیرد. از آنجایی که عیسی کارهای باارزش‌تری برای انجام دارد، از شما خواسته تا این کار را برایش بکنید.

در ورودی برنامه، درخت دودویی به صورت آرایه به شما داده‌می‌شود که مقدار ریشهٔ آن a_0 است؛ مقدار بچهٔ چپ ریشه، a_1، و مقدار بچهٔ راست آن a_2 است. به طور کلی برای رأس با مقدار a_k، مقدار بچهٔ چپ آن a_{2k+1}، و مقدار بچهٔ راست آن a_{2k+2} است.

ورودی:

در خط اول، n، تعداد رئوس درخت داده‌می‌شود.
سپس در خط دوم n عدد که با فاصله جدا شده‌اند داده‌می‌شود (a_0 تا a_{n-1}) که مقادیر رئوس درخت هستند.

خروجی:

در تنها سطر خروجی، حداقل تعداد تعویض‌های لازم برای تبدیل درخت به یک درخت جستجوی دودویی را چاپ نمایید.

محدودیت‌ها:

$1 \le n \le 10^5$
$0 \le a_i \le 10^5$

ورودی و خروجی نمونه:

تست اول

stdin	stdout
3 1 2 3	1

پاسخ: پاسخ نمونه.

۲ تمرینات عملی

سؤال ۸. [۲۶ نمره] در پیاده‌سازی مرتب‌سازی سریع، هر بار پس از تبدیل زیرآرایهٔ ورودی تابع به زیرآرایه‌های چپ و راست و عنصر لولا و صدا زدن دوبارهٔ تابع مرتب‌سازی سریع روی زیرآرایه‌های چپ و راست (به ترتیب)، زیرآرایهٔ ورودی را چاپ کنید. عنصر لولا را اولین عنصر زیرآرایهٔ ورودی در نظر بگیرید.
دقت کنید که هنگام تقسیم زیرآرایه به دو زیرآرایهٔ چپ و راست، باید اعضای کمتر از لولا باید با حفظ ترتیب به اول زیرآرایه منتقل شوند و زیرآرایهٔ سمت چپ را بسازند و به طور مشابه ترتیب اولیهٔ اعضای بزرگتر از لولا نیز در زیرآرایهٔ سمت راست حفظ شود.

ورودی:

در خط اول، n، طول آرایه داده‌می‌شود.
سپس در خط دوم n عدد متمایز که با فاصله جدا شده‌اند داده‌می‌شود (a_0 تا a_{n-1}) که مقادیر خانه‌های آرایه به ترتیب هستند.

خروجی:

در سطرهای خروجی، باید زیرآرایهٔ مرتب‌شدهٔ ورودی تابع مرتب‌سازی سریع به ترتیب خروج از توابع، هر کدام در یک سطر چاپ شود.

محدودیت‌ها:

$1 \leq n \leq 3 \times 10^3$
$-1000 \leq a_i \leq 10^9$

ورودی و خروجی نمونه:

تست اول

stdin	**stdout**
7 5 8 1 3 7 9 2	2 3 1 2 3 7 8 9 1 2 3 5 7 8 9

پاسخ: پاسخ نمونه.

سؤال ۹. [۴۱ نمره] به یک رشته، زیبا می‌گوییم، اگر تعداد aها، bها و cهای آن برابر باشد؛ مثلاً "abc"، "$aabbcc$"، "$dabc$" و "" زیبا هستند. در این سؤال شما باید تعداد زیررشته‌های زیبای یک رشته از حروف کوچک انگلیسی را پیدا کنید.

ورودی:

در تنها خط ورودی، رشته‌ای که باید تعداد زیررشته‌های زیبای آن را پیدا کنید، داده‌می‌شود.

خروجی:

در تنها خط خروجی، باید تعداد زیررشته‌های زیبای رشتهٔ ورودی را چاپ کنید.

محدودیت‌ها:

$۱ \leq |s| \leq ۱۰^{۷}$
$۰ \leq |a|, |b|, |c| \leq ۱۰۰۰$

ورودی و خروجی نمونه:

تست اول

stdin	**stdout**
abacbcba	5

تست دوم

stdin	**stdout**
dbacd	6

پاسخ: پاسخ نمونه.

موفق باشید